

DATABASES COURSE 10: INDEXING

1. Indexes
2. Partitioned tables

1. Indexes

The access to data can be made sequential, direct or dynamic (sequential and direct access).

The sequential access on a table, assumes crossing the whole table, meaning accessing each record in the loading order.

Direct access is made using indexing keys.

Indexes are logically and physically independent of the data in the associated table. Because they are separate independent objects, they require some storage space. Indexes can be created or dropped without affecting the tables in the database, the applications, or other indexes. The database automatically takes care of the indexes when an insert, update or delete operation is made on the table that has indexes. If an index is dropped, the applications will not be affected, but some queries might work slower.

A table can have any number of indexes, but if it has too many it can take a lot of time to modify it. When rows are inserted or deleted, all indexes on the table must be updated as well. Also, when a column is updated, all indexes that contain the column must be updated. There has to be a balance between the speed of retrieving data from a table and the speed of updating the table. There also are the extreme cases of historical data, where we have very few updates, so we can use lots of indexes, and of big data, where we have lots of updates so we need few indexes.

Indexes can be associated to tables or clusters.

Indexing guide:

1. Decide the necessity of using an index: if the time wasted with processing a query is reduced or if ordering the data is necessary.
2. Choosing an indexing key: the attributes in WHERE clause are candidates for the key.
3. Keys from multiple attributes: many separated attributes (multiple keys) or concatenated attributes (composed key), in a certain order.
4. Building clusters: advantage for requests, disadvantage for updates.
5. Indexing type: trees (B, R, QUAD), hash (for join requests), spatial (for spatial requests), LOB (for LOB data), etc.
6. Evaluating the cost of maintaining the indexes:
 - they should be deleted if the requests are rare => the storage space is cleared
 - they should be kept if the requests are often => access time is reduced

Indexes are automatically created when a unique constraint is stated (primary key included). The system names them and they can be simple or composed indexes depending on the key type. Also, indexes can be created by the user when creating the unique constraint, or can just be named.

B-tree indexes in Oracle:

B-tree is the most common used index in Oracle DBMS and offers good results for simple requests.

[[create index +unique constraint! -> automatically created for PK or unique constraints]]

```
CREATE [UNIQUE] INDEX index_name ON table_name (columns);
```

All indexes on a table can be seen by running the command:

```
SELECT * FROM USER_INDEXES WHERE TABLE_NAME='name_of_the_table';
```

All indexes which belong to the logged user:

```
SELECT * FROM USER_INDEXES;
```

Function based indexes are used when certain expressions containing functions are frequently used in the same way in different queries.

For example, if we often use the following select:

```
SELECT * FROM employees WHERE upper(substr(name,1,1))='A';
```

Then we should have had defined the following index:

```
CREATE INDEX emp_idx ON employees(upper(substr(name,1,1)));
```

Indexes can be unique or non-unique. Unique indexes guarantee that no two rows of a table have duplicate values in the key column (or columns).

```
CREATE TABLE order(  
    client VARCHAR2(20),  
    order_date DATE,  
    CONSTRAINT c1 UNIQUE (client, order_date)  
    USING INDEX (create unique index idx1 on order(client, order_date)));
```

In certain cases, we can modify the optimizing execution plan by using hints after the SELECT key word:

```
/*+NO_INDEX(table_name [index_name])*/
```

forces the optimizer no to use the specified indexes

```
/*+INDEX(table_name index_name)*/
```

tells the optimizer to use the specified index

2. Partitioned tables

Partitioned tables allow data to be devised into smaller units called partitions, or even subpartitions. Indexes can be partitioned in a similar way. Each partition is stored in its own segment and can be managed individually. It can function independently of the other partitions, thus providing a structure that can be better managed regarding availability and performance.

Partitions and subpartitions of a table or index all share the same logical attributes. For example, all partitions in a table share the same column and constraint definitions, and all partitions of an index share the same index options.

In Oracle, the following partitions can be implemented, and will be exemplified:

- Range partitions
- Hash partitions
- List partitions
- Composite range-hash partitions
- Composite range-list partitions

2.1. Range Partitions

We use range partitioning when we can define intervals of values between which all values in a column can be placed. If partitioning by range causes partitions to vary a lot in size because of unequal distribution, one of the other methods of partitioning may be used.

When creating range partitions, there must be specified:

- ☐ the range partitioning method
- ☐ the partitioning columns
- ☐ the partition bounds

Ex. Creating a table with 4 partitions for each type of employee considering its hiring date.

```
CREATE TABLE employees
  ( employee_id number primary key,
    hire_year  number(4,0) not null,
    hire_month number(2,0) not null,
    hire_day   number(2,0) not null,
    country varchar2(3))
PARTITION BY RANGE (hire_year, hire_month, hire_day)
( PARTITION seniors VALUES LESS THAN (2000, 01, 01)
  TABLESPACE tsa,
  PARTITION avg_emp VALUES LESS THAN (2008, 06, 01)
  TABLESPACE tsb,
  PARTITION juniors VALUES LESS THAN (2016, 01, 01)
  TABLESPACE tsc,
  PARTITION newbie VALUES LESS THAN (2016, 07, 20)
  TABLESPACE tsd );
```

2.2. Hash Partitions

Hash partitioning is used for evenly distributing data in a specified number of partitions.

When creating hash partitions, there must be specified:

- ☐ the hash partitioning method
- ☐ the partitioning columns
- ☐ the number of partitions

Ex. Creating a table with 3 even partitions.

```
CREATE TABLE customers
  (customer_id number,
   name  varchar2(25),
   address varchar2(25))
PARTITION BY HASH(customer_id)
( PARTITION part1 TABLESPACE tbs_1
, PARTITION part2 TABLESPACE tbs_2
, PARTITION part3 TABLESPACE tbs_3);
```

Or, if we let the system name the partitions:

```
CREATE TABLE customers
  (customer_id number,
   name  varchar2(25),
   address varchar2(25))
PARTITION BY HASH(customer_id)
```

PARTITIONS 3
STORE IN (tbs_1,tbs_2,tbs_3);

2.3. List Partitions

We use list partitioning when we have a list of discrete values for the partitioning column that is not so big. We place value by value in groups, each group corresponding to a partition.

When creating list partitions, there must be specified:

- ☐ the list partitioning method
- ☐ the partitioning columns
- ☐ the list of values for each partition

Ex. Creating a table with 3 partitions for each order mode: on-line, cash, cash on-delivery.

```
CREATE TABLE orders
  (order_id number primary key,
   order_date date,
   customer_id number,
   order_mode varchar2(8))
PARTITION BY LIST (order_mode)
(PARTITION order_on_line VALUES ('online', 'web'),
 PARTITION order_cash VALUES ('eur', 'ron', 'dollar'),
 PARTITION order_on_delivery VALUES ('delivery'));
```

2.4. Composite Range-Hash Partitions

Range-hash partitioning use the range method for creating partitions, and within each partition, subpartitions are creating using the hash method. These composite partitions are ideal for historical data.

When creating range-hash partitions, there must be specified:

- ☐ the range partitioning method
- ☐ the partitioning columns
- ☐ the partion limits
- ☐ the hash partitioning method for the subpartitions
- ☐ the subpartitioning column
- ☐ the number of subpartitions for each partition

Ex. of Composite Range-Hash Partitions for CUSTOMERS table.

```
CREATE TABLE customers
  (customer_id number,
   name  varchar2(25),
   address  varchar2(25))
PARTITION BY RANGE (customer_id)
SUBPARTITION BY HASH(name)
SUBPARTITIONS 5
(PARTITION part1 VALUES LESS THAN (100),
 PARTITION part2 VALUES LESS THAN (500),
 PARTITION part3 VALUES LESS THAN (MAXVALUE));
```

2.5. Composite Range-List Partitions

Range-list partitioning use the range method for creating partitions, and within each partition, subpartitions are creating using the list method. These composite partitions are also ideal for historical data.

When creating range-list partitions, there must be specified:

- ☐ the range partitioning method
- ☐ the partitioning columns
- ☐ the partion limits
- ☐ the list partitioning method for the subpartitions
- ☐ the subpartitioning column
- ☐ the list of values for each subpartition

Ex. Creating a table with 4 partitions for each type of employee considering its hiring date.

```
CREATE TABLE employees
( employee_id number primary key,
  hire_date date,
  country varchar2(3))
PARTITION BY RANGE (hire_date)
SUBPARTITION BY LIST (country)
(PARTITION seniors VALUES LESS THAN (to_date('2000,01,01', 'YYYY,MM,DD'))
  (SUBPARTITION eu1 VALUES ('ro','fr','eng'),
   SUBPARTITION am1 VALUES ('usa','mex'),
   SUBPARTITION asia1 VALUES ('ru','jpn','ind')),
 PARTITION avg_emp VALUES LESS THAN (to_date('2008,06,01', 'YYYY,MM,DD'))
  (SUBPARTITION eu2 VALUES ('ro','fr','eng'),
   SUBPARTITION am2 VALUES ('usa','mex'),
   SUBPARTITION asia2 VALUES ('ru','jpn','ind')),
 PARTITION juniors VALUES LESS THAN (to_date('2016,01,01', 'YYYY,MM,DD'))
  (SUBPARTITION eu3 VALUES ('ro','fr','eng'),
   SUBPARTITION am3 VALUES ('usa','mex'),
   SUBPARTITION asia3 VALUES ('ru','jpn','ind')),
 PARTITION newbie VALUES LESS THAN (to_date('2016,07,20', 'YYYY,MM,DD'))
  (SUBPARTITION eu4 VALUES ('ro','fr','eng'),
   SUBPARTITION am4 VALUES ('usa','mex'),
   SUBPARTITION asia4 VALUES ('ru','jpn','ind')) );
```

The general syntax for a partitioned table can be synthesized like this:

```
CREATE TABLE table_name (columns types)
PARTITION BY LIST/RANGE/HASH (columns)
(PARTITION p1 VALUES.....
 PARTITION p2 VALUES.....
 .....);
```

Maintaining Partitions

- https://docs.oracle.com/cd/E11882_01/server.112/e25523/part_admin002.htm
- For HASH-partitioned tables:

- ALTER TABLE pcustomers COALESCE PARTITION;
- ALTER TABLE pcustomers ADD PARTITION;
- For LIST-partitioned tables:
 - ALTER TABLE p_orders MODIFY PARTITION order_cash
ADD VALUES ('pound', 'yen');
- ALTER TABLE employees DROP PARTITION newbie;